

Lab 22 — A Real Display (SPI TFT and DMA)

ESP32-S3 FIRMWARE LAB SERIES · PHASE 3 · GAP-CLOSERS · GAP-CLOSER

Estimated time	3 hrs
Proves	DMA buffer ownership, real ping-pong
Prerequisites	Lab 9 (SPI).
Beyond the board	An ST7789 SPI TFT and seven jumper wires.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§22.1 objective**, **§22.2 parts and wiring**, **§22.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

KEY POINT

How this lab is structured

Concept, then **Phase 0 configure and wire**, **Phase 1 the complete code**, walkthrough, **Phase 2 run**, **Phase 3 prove it**, worked extensions.

The lab implements **real ping-pong double buffering**, states the buffer-ownership invariant explicitly, and ends with a deliberate corruption test — because a renderer that merely *looks* double-buffered is one of the easiest mistakes to ship.

22.1 Objective & TM4C bridge

Drive a colour TFT over SPI using the `esp_lcd` component, pushing pixels with DMA. By the end you can put live sensor data on a physical screen, which is the single most demonstrable thing you can carry to a career fair.

TM4C BRIDGE

You have probably driven a display before. The standard bare-metal ADC-plus-LCD lab drives an HD44780 character LCD in 4-bit nibble mode, toggling the **RS** line to distinguish command bytes from data bytes, strobing **E**, and hand-timing delays between writes. Two things carry straight over. The **DC** pin here is your RS pin under a different name — same job, same idea. And the initialisation sequence is the same concept: a documented series of command bytes the controller needs before it will display anything.

What is genuinely new is the *kind* of display. HD44780 is a character device: you send it a byte, it draws a glyph from its own font ROM. The ST7789 is a **framebuffer** device: nothing has a shape until you specify the colour of every individual pixel. That changes the problem from "send 32 characters" to "move 115 KB," which is exactly why DMA appears.

22.2 Parts & wiring

Item	Status	Notes
ST7789 240×240 IPS SPI TFT	~\$10	1.3" or 1.54" is ideal. Get one that breaks out a CS pin — some bare 240×240 modules omit it, which complicates sharing the bus.
7 jumper wires	Required	—

Display pin	ESP32-S3	Notes
VCC	3V3	—
GND	GND	Connect first.
SCLK	GPIO12	Same SPI bus as Lab 9
MOSI / SDA	GPIO11	—
CS	GPIO10	—
DC	GPIO14	Data/command select — your old RS pin
RST	GPIO15	Panel reset
BL / LED	3V3	Backlight. Some modules leave it off by default.

WARNING**A dark screen is usually the backlight**

If the panel shows nothing at all and the code reports no errors, check the backlight pin before debugging anything else. A live panel with the backlight off looks identical to a dead one.

22.3 How it works

A graphical display is the first peripheral where *memory* and *throughput* become the design problem rather than an afterthought. Both follow from one number.

A framebuffer display, versus the character LCD you already drove

On a bare-metal course you drive an HD44780 character LCD. You send it the byte `0x41` and an "A" appears, because the controller has a font ROM inside it and knows what letters look like. You could display 32 characters and nothing else.

The ST7789 knows nothing about letters. It is a **framebuffer** device: a grid of pixels, each of which has a colour, and *nothing has a shape until you specify every pixel individually*. Drawing the letter "A" means working out which pixels are dark. That is more work, and it is also complete freedom — any font, any size, any graph, any image.

The controller has its own memory, called **GRAM**, holding what is currently on the glass. Your job is to push pixel data into it over SPI.

The number that drives everything

RGB565: one pixel packed into 16 bits

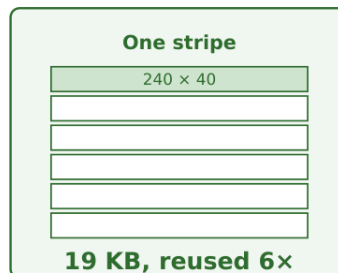
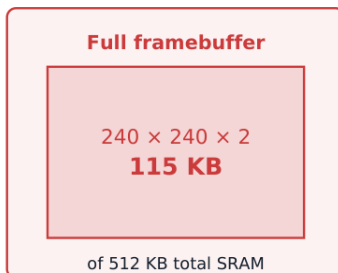


Green gets the extra bit because the eye is most sensitive to green. $5 + 6 + 5 = 16$ bits = 2 bytes per pixel.

So the three primaries are: `0xF800` = all 5 red bits set, `0x07E0` = all 6 green, `0x001F` = all 5 blue.

Work one of those out against the layout above once, and the format stops being opaque.

Why the code pushes stripes instead of a whole frame



The trade

Stripes: 6× less RAM, but you cannot read back what is already on screen.

Full buffer: draw anywhere, compose freely, then send it all at once.

A full buffer fits comfortably in the 8 MB of PSRAM — slower, but roomy.

Either way the buffer must be DMA-capable, which is why it comes from `heap_caps_malloc(..., MALLOC_CAP_DMA)` rather than being declared as a plain array. Hand the DMA engine an ordinary buffer and the transfer fails or corrupts.

Figure 22.1. Top: how one pixel packs into 16 bits, and where the three primary constants come from. Bottom: why the code pushes stripes — a full frame is 115 KB of the chip's 512 KB, while one stripe is 19 KB reused six times.

RGB565 bit layout, and why the display is pushed in stripes.

One pixel packed into 16 bits: 5 bits of Red (bits 15 down to 11), 6 bits of Green (bits 10 down to 5), 5 bits of Blue (bits 4 down to 0). Green gets the extra bit because the eye is most sensitive to green. $5 + 6 + 5 = 16$ bits = 2 bytes per pixel.

So the three primaries are: `0xF800` = all 5 red bits set, `0x07E0` = all 6 green bits set, `0x001F` = all 5 blue bits set.

Why the code pushes stripes instead of a whole frame:

Approach	Size	Note
Full framebuffer	240 x 240 x 2 = 115 KB	of only 512 KB total SRAM
One stripe	240 x 40 x 2 = 19 KB, reused 6 times	6x less RAM

The trade: stripes use 6x less RAM but you cannot read back what is already on screen. A full buffer lets you draw anywhere and compose freely, then send it all at once, and it fits comfortably in the 8 MB of PSRAM -- slower, but roomy.

Either way the buffer must be DMA-capable, which is why it comes from `heap_caps_malloc(..., MALLOC_CAP_DMA)` rather than being declared as a plain array. Hand the DMA engine an ordinary buffer and the transfer fails or corrupts.

240 × 240 is **57,600 pixels**. In **RGB565** — 16 bits per pixel, split 5 red, 6 green, 5 blue — that is 115,200 bytes. Green gets the extra bit because human vision is most sensitive to it, which is a nice piece of engineering pragmatism sitting inside a data format.

115 KB against 512 KB of SRAM is a serious allocation, and it forces the first real decision: hold a whole framebuffer, or push the screen in pieces?

This lab pushes **stripes**: one buffer of 240 × 40 pixels, about 19 KB, filled and sent six times to cover the screen. Six times less RAM, at the cost of not being able to read back what is already displayed. The alternative — a full framebuffer in PSRAM, where 115 KB is nothing against 8 MB — is the \$22.7 exercise, and it is how real graphical firmware is usually built.

Why DMA stops being optional

Pushing 115 KB one byte at a time, with the CPU issuing each transfer, would consume the core and stutter visibly. This is the same argument as Lab 20, now with an obvious visual payoff: the DMA engine moves each stripe while the CPU prepares the next one.

That overlap is where the throughput actually comes from, and it is why `.trans_queue_depth` is set to 10 rather than 1 — the driver can have several transfers queued so the pipeline never drains.

The constraint DMA imposes is the one from Lab 20: the buffer must live in **DMA-capable memory**, so it comes from `heap_caps_malloc(..., MALLOC_CAP_DMA)` rather than being declared as a plain array.

The DC pin, which is your old RS pin

SPI carries bytes with no notion of what they mean, so the panel needs a way to tell a *command* ("set the drawing window to these coordinates") from *data* ("here are 19 KB of pixels"). A single extra GPIO does it: the **DC** line is low while a command byte is clocked out and high for data.

This is exactly the **RS** line you toggled by hand on the HD44780, under a different name and doing the same job. Here the `esp_lcd` transport layer drives it for you, which is the main reason that layer exists.

Two layers, and why the split is worth noticing

`esp_lcd` separates the job cleanly, and it is a good example of driver layering:

Layer	Knows about	Knows nothing about
Panel IO (<code>esp_lcd_new_panel_io_spi</code>)	Moving commands and pixels over SPI with DMA, and toggling DC	Any particular display controller
Panel driver (<code>esp_lcd_new_panel_st7789</code>)	The ST7789's specific command set and init sequence	How bytes physically get there

Swap the panel driver and the same transport drives an ILI9341. Swap the IO layer and the same ST7789 driver works over a parallel bus instead of SPI. That is what a well-factored driver stack looks like, and it is worth being able to describe.

The gotchas that will actually cost you time

The backlight. Many modules leave it off by default. A live panel with no backlight is visually identical to a dead one, so check that pin before debugging anything else.

Colour inversion. Most ST7789 modules need `esp_lcd_panel_invert_color(panel, true)`. If red shows as cyan and blue as yellow, that is the line — a one-argument experiment, not a debugging session.

Offsets. Many 240×240 modules are physically a 240×320 panel with part of it unused, so the visible area starts at a non-zero offset. If your image is shifted or shows a coloured band at one edge, `esp_lcd_panel_set_gap()` is the fix.

Wire length at 40 MHz. Long breadboard jumpers have capacitance and act as antennas. At 1 MHz nothing cares; at 40 MHz you will see torn or corrupted output. Short, direct wiring is a real requirement at speed — and if you see tearing, dropping to 20 MHz to confirm the diagnosis is the fastest test.

22.4 Phase 0 — configure and wire

0.1 — project files

```
// sdkconfig.defaults
CONFIG_IDF_TARGET="esp32s3"
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
CONFIG_FREERTOS_HZ=1000
```

```
// top-level CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab22_display)
```

```
// main/CMakeLists.txt
idf_component_register(SRCS "main.c" INCLUDE_DIRS ".")
```

0.2 — wiring, seven wires

Display	ESP32-S3	Note
VCC	3V3	not 5V
GND	GND	
SCL / SCK	GPIO12	SPI clock
SDA / MOSI	GPIO11	data to the panel
CS	GPIO10	chip select
DC / RS	GPIO14	command vs data — your HD44780 RS line
RST	GPIO15	hardware reset
BLK / LED	3V3	backlight — check this first

WARNING

Two things that will cost you an hour each

The backlight. Many modules leave it off. A live panel with no backlight is visually identical to a dead one. Tie BLK to 3V3 before debugging anything else.

Wire length at 40 MHz. Long breadboard jumpers have capacitance and act as antennas. At 1 MHz nothing cares; at 40 MHz you get torn or corrupted output. Keep them short and direct. If you see tearing, drop `pclk_hz` to 20 MHz to confirm the diagnosis before chasing your code.

0.3 — the memory arithmetic

```
full framebuffer = 240 x 240 x 2 bytes = 115,200 B = 112.5 KB   of 512 KB SRAM
one stripe       = 240 x 40 x 2 bytes = 19,200 B = 18.8 KB
stripes per frame = 240 / 40           = 6                (divides evenly -- no partial stripe)
TWO stripes      = 2 x 18.8 KB          = 37.5 KB         <- what this lab uses
```

40 divides 240 exactly. A value like 50 would leave a partial final stripe the loop does not handle, which is the kind of thing that renders five sixths of a screen correctly and looks like a driver bug.

22.5 Phase 1 — the complete main.c

KEY POINT

The ownership invariant this code maintains

A buffer is written only while the application owns it, and ownership returns only when the transfer for that exact buffer has completed.

Buffer identity is tracked by round-robin index. That is valid *because transfers on a single panel IO handle complete in FIFO order* — they are queued to one SPI host and serviced in sequence. The counting semaphore therefore always releases the oldest outstanding buffer, which is the one the round-robin is about to reuse. **If you ever queue to two panel handles, or the driver stops guaranteeing FIFO, this assumption breaks** and you would need explicit per-buffer tokens.

```
// =====
// Lab 22 -- ST7789 over SPI with DMA and REAL ping-pong buffering
// =====
#include <stdio.h>
#include <string.h>
#include <inttypes.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "driver/spi_master.h"
#include "driver/gpio.h"
#include "esp_lcd_panel_io.h"
#include "esp_lcd_panel_vendor.h"
#include "esp_lcd_panel_ops.h"
#include "esp_heap_caps.h"
#include "esp_timer.h"
#include "esp_log.h"

#define PIN_SCLK    GPIO_NUM_12
#define PIN_MOSI    GPIO_NUM_11
#define PIN_CS      GPIO_NUM_10
#define PIN_DC      GPIO_NUM_14
#define PIN_RST     GPIO_NUM_15

#define H_RES       240
#define V_RES       240
#define STRIPE      40                // 240/40 = 6, divides evenly
#define STRIPE_PX   (H_RES * STRIPE)
#define NBUF        2                // set to 1 for the corruption test

static const char *TAG = "lab22";

static esp_lcd_panel_io_handle_t io;
static esp_lcd_panel_handle_t panel;

static uint16_t      *buf[NBUF];
static SemaphoreHandle_t buf_free;           // counts buffers the APP owns
static volatile uint32_t g_done;            // ISR only

// RGB565: 5 bits red (15..11), 6 green (10..5), 5 blue (4..0).
// Green gets the extra bit because the eye is most sensitive to green.
static inline uint16_t rgb565(uint8_t r, uint8_t g, uint8_t b)
{
    return (uint16_t)((((r & 0xF8) << 8) | ((g & 0xFC) << 3) | (b >> 3)));
}

// A DIFFERENT colour per stripe, on purpose. With six identical stripes the
// single-buffer bug is invisible; with six different ones it is obvious.
static uint16_t band_colour(int band, int frame)
{
    static const uint16_t pal[6] = {
        0xF800, 0xFD20, 0xFFE0, 0x07E0, 0x001F, 0x780F, // R O Y G B V
    };
    return pal[(band + frame) % 6];
}

// ----- transfer-done callback
// This is the ONLY thing that may return a buffer to the application.
```

```

// Not a delay, not a guess -- the hardware telling you it is finished.
static bool IRAM_ATTR on_color_done(esp_lcd_panel_io_handle_t h,
                                   esp_lcd_panel_io_event_data_t *e, void *ctx)
{
    BaseType_t hp = pdFALSE;
    g_done++;
    xSemaphoreGiveFromISR(buf_free, &hp);
    return hp == pdTRUE;
}

// ----- bring-up
static void display_init(void)
{
    spi_bus_config_t bus = {
        .sclk_io_num    = PIN_SCLK,
        .mosi_io_num    = PIN_MOSI,
        .miso_io_num    = -1,           // write-only: no MISO wire at all
        .quadwp_io_num  = -1,
        .quadhd_io_num  = -1,
        .max_transfer_sz = STRIPE_PX * sizeof(uint16_t), // must cover one stripe
    };
    ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &bus, SPI_DMA_CH_AUTO));

    esp_lcd_panel_io_spi_config_t io_cfg = {
        .dc_gpio_num    = PIN_DC,
        .cs_gpio_num    = PIN_CS,
        .pclk_hz        = 40 * 1000 * 1000, // 40x Lab 9 -- DMA does the pushing
        .lcd_cmd_bits   = 8,
        .lcd_param_bits = 8,
        .spi_mode        = 0,             // CPOL 0, CPHA 0, as Lab 9
        .trans_queue_depth = 10,
    };
    ESP_ERROR_CHECK(esp_lcd_new_panel_io_spi(
        (esp_lcd_spi_bus_handle_t)SPI2_HOST, &io_cfg, &io));

    // Register the callback BEFORE any transfer can complete.
    esp_lcd_panel_io_callbacks_t cbs = { .on_color_trans_done = on_color_done };
    ESP_ERROR_CHECK(esp_lcd_panel_io_register_event_callbacks(io, &cbs, NULL));

    esp_lcd_panel_dev_config_t pcfg = {
        .reset_gpio_num = PIN_RST,
        .rgb_ele_order  = LCD_RGB_ELEMENT_ORDER_RGB, // was .color_space pre-5.0
        .bits_per_pixel = 16,
    };
    ESP_ERROR_CHECK(esp_lcd_new_panel_st7789(io, &pcfg, &panel));

    ESP_ERROR_CHECK(esp_lcd_panel_reset(panel));
    ESP_ERROR_CHECK(esp_lcd_panel_init(panel));
    ESP_ERROR_CHECK(esp_lcd_panel_invert_color(panel, true)); // most ST7789 modules
    ESP_ERROR_CHECK(esp_lcd_panel_disp_on_off(panel, true));
}

// ----- render
static void display_task(void *arg)
{
    (void)arg;

    buf_free = xSemaphoreCreateCounting(NBUF, NBUF); // all start owned by us
    configASSERT(buf_free);
}

```

```

for (int i = 0; i < NBUF; i++) {
    buf[i] = heap_caps_malloc(STRIPE_PX * sizeof(uint16_t), MALLOC_CAP_DMA);
    if (!buf[i]) { // CHECK. 19 KB DMA-capable can fail.
        ESP_LOGE(TAG, "buffer %d alloc failed", i);
        vTaskDelete(NULL);
    }
}
ESP_LOGI(TAG, "%d buffers x %d B DMA-capable", NBUF,
          (int)(STRIPE_PX * sizeof(uint16_t)));

int idx = 0, frame = 0;
int64_t t_fill = 0, t_frame0 = esp_timer_get_time();

for (;;) {
    for (int y = 0, band = 0; y < V_RES; y += STRIPE, band++) {
        // Wait for a buffer WE own. Not a delay -- an event.
        // With both in flight this blocks, so we physically cannot
        // overwrite a buffer the hardware is still reading.
        xSemaphoreTake(buf_free, portMAX_DELAY);

        uint16_t *b = buf[idx]; // claim it, but do NOT advance yet

        int64_t f0 = esp_timer_get_time();
        uint16_t col = band_colour(band, frame);
        for (int i = 0; i < STRIPE_PX; i++) b[i] = col;
        t_fill += esp_timer_get_time() - f0;

        esp_err_t err = esp_lcd_panel_draw_bitmap(panel, 0, y,
                                                  H_RES, y + STRIPE, b);
        if (err == ESP_OK) {
            // ONLY now is the buffer genuinely in flight, so only now may
            // the round-robin move on. Advancing before this point would
            // desynchronise the index from what the completion callback
            // is about to release, and the next frame would write into a
            // buffer the hardware is still reading.
            idx = (idx + 1) % NBUF; // valid: transfers complete FIFO
        } else {
            // Enqueue failed, so no completion will EVER fire for this
            // buffer. Return the permit or we leak it and eventually
            // deadlock, and leave idx alone so we retry the same slot.
            ESP_LOGE(TAG, "draw_bitmap: %s", esp_err_to_name(err));
            xSemaphoreGive(buf_free);
        }
        // b is now IN FLIGHT. Do not touch it. on_color_done releases it.
    }

    if (++frame % 60 == 0) {
        int64_t dt = esp_timer_get_time() - t_frame0;
        ESP_LOGI(TAG, "60 frames in %lld ms -> %.1f fps | fill %lld ms | done=%" PRIu32,
                  dt / 1000, 60.0 * 1e6 / (double)dt, t_fill / 1000, g_done);
        t_fill = 0; t_frame0 = esp_timer_get_time();
    }
}

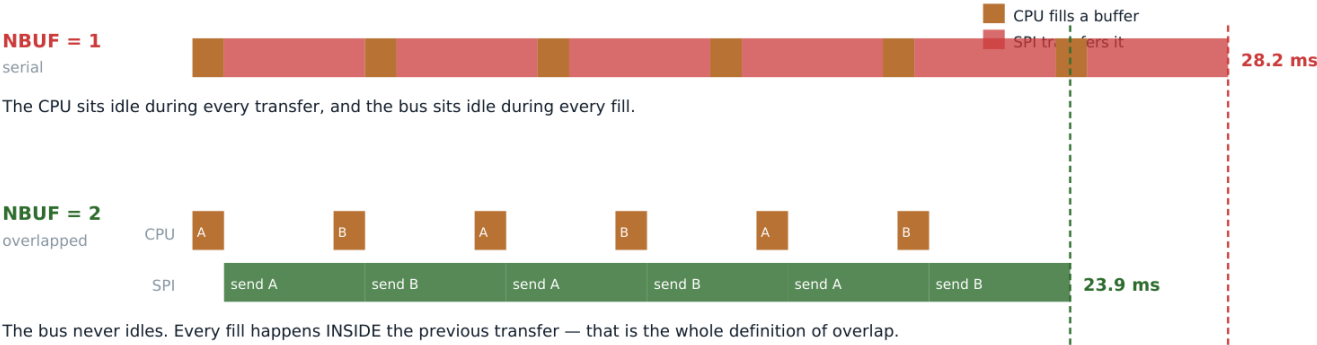
void app_main(void)
{
    display_init();
}

```

```
xTaskCreate(display_task, "disp", 4096, NULL, 5, NULL);  
}
```

What the second buffer actually buys you

One 240×40 stripe: CPU fill 0.86 ms, SPI transfer 3.84 ms at 40 MHz. Six stripes make a frame.



The arithmetic [DERIVED, with one assumption]

serial = $(0.86 + 3.84) \times 6 = 28.2 \text{ ms/frame} \rightarrow 35.5 \text{ fps}$
overlap = $0.86 + (3.84 \times 6) = 23.9 \text{ ms/frame} \rightarrow 41.8 \text{ fps}$
Transfer time is derived from the bus: $19,200 \text{ B} \times 8 \div 40 \text{ MHz}$. Fill time is ASSUMED from the lab's own `t_fill` instrumentation (~310 ms per 60 frames). Measure yours — do not quote these.

What to look for when you measure it

Not simply “two is faster.” The CPU does IDENTICAL work in both rows — six fills either way — so your `t_fill` total should be about the SAME while the frame rate rises. That is the signature of overlap.
If fill time also changes, something else moved and the comparison is not clean.
And if the two rows come out close, your fill is simply cheap next to your transfer. That is a real result.

Figure 22.2. [DERIVED] The same six stripes, drawn twice. With one buffer the CPU idles during every transfer and the bus idles during every fill. With two, every fill happens *inside* the previous transfer.

Ping-pong buffering: what the second buffer actually buys. [DERIVED] Two timelines of the same six stripes. One 240x40 stripe costs 0.86 ms of CPU fill and 3.84 ms of SPI transfer at 40 MHz.

NBUF = 1 (serial): fill, transfer, fill, transfer, six times in sequence. Total **28.2 ms per frame -> 35.5 fps**. The CPU sits idle during every transfer and the bus sits idle during every fill.

NBUF = 2 (overlapped): two rows. The CPU row shows six short fills alternating between buffers A and B. The SPI row shows six transfers running back to back with no gaps, each starting as soon as the previous ends. Total **23.9 ms per frame -> 41.8 fps**. Every fill happens inside the previous transfer.

The arithmetic:

	Calculation	Result
serial	$(0.86 + 3.84) \times 6$	28.2 ms/frame, 35.5 fps
overlapped	$0.86 + (3.84 \times 6)$	23.9 ms/frame, 41.8 fps
pixel-only ceiling	$921,600 \text{ bits} / 40 \text{ MHz}$	43.4 fps

Transfer time is derived from the bus: 19,200 bytes x 8 / 40 MHz. **Fill time is ASSUMED** from the lab's own `t_fill` instrumentation (about 310 ms per 60 frames). Measure your own — do not quote these.

What to look for when you measure it: **not simply "two is faster."** The CPU does identical work in both rows — six fills either way — so your `t_fill` total should stay about the same while the frame rate rises. That is the signature of overlap. If fill time also changes, something else moved and the comparison is not clean. And if the two rows come out close, your fill is simply cheap next to your transfer, which is a real result rather than a failed experiment.

The bus is the bottleneck here, so a third buffer would gain nothing — no amount of buffering makes a wire faster.

The timeline is worth studying before the code, because it shows precisely what "overlap" means and what it does not.

The CPU does identical work in both rows. Six fills either way, 0.86 ms each. Nothing got faster. What changed is *when* that work happens — between transfers, or during them.

The bus is the bottleneck here, and that sets the ceiling. Transfer is 3.84 ms against 0.86 ms of fill, so once you overlap, the frame time collapses to almost exactly six transfers back to back. Adding a third buffer would gain you nothing at all: the bus is already saturated, and no amount of buffering makes a wire faster.

That last point is the one worth carrying. **Double buffering hides the cheaper of two costs behind the more expensive one.** If your fill were the expensive side — a heavy per-pixel computation rather than a constant colour — the roles would reverse and the CPU would become the limit. Knowing which side you are on decides whether the next optimisation belongs in the code or in the clock rate.

22.6 Code walkthrough, the parts that matter

`.miso_io_num = -1`

A display is **write-only** — you send pixels and never read them — so there is no MISO wire at all, and `-1` is the driver's convention for "no pin." One fewer wire to route, which matters on a real board.

`.max_transfer_sz = STRIPE_PX * sizeof(uint16_t)`

Caps the largest single DMA transfer and must be at least one stripe — 19,200 bytes here, against the 32 bytes Lab 9 needed. Set it too small and `draw_bitmap` fails at run time with an error that does not obviously point back here. Note it is *computed* from the same constants, so raising `STRIPE` raises this automatically.

`esp_lcd_panel_io_register_event_callbacks` BEFORE any drawing

The same initialisation-order discipline as Lab 8's interrupt handler. Register first, so no transfer can complete before there is somebody to handle it.

`xSemaphoreCreateCounting(NBUF, NBUF)`

A **counting** semaphore initialised to `NBUF` is a permit pool: taking one means "I have claimed a buffer," giving one from the completion callback means "a buffer just came back." **The count is the number of buffers the application owns.**

Two properties fall out. The wait is *exact* — the CPU proceeds the instant a transfer finishes, no sooner. And it is **self-limiting**: with both buffers in flight the take blocks, so you physically cannot queue a third transfer or overwrite a buffer the hardware is still reading.

Contrast Lab 7's *binary* semaphore, whose count saturates at one. Here the count carries real information, so counting is required rather than stylistic.

idx = (idx + 1) % NBUF

Buffer identity by round-robin. Valid **only because transfers on one panel IO complete in FIFO order** — they are queued to a single SPI host and serviced in sequence, so the semaphore always releases the oldest outstanding buffer, which is exactly the one this index is about to reuse. State that invariant when you describe this design; it is the difference between a justified assumption and a lucky one.

the enqueue failure path

Easy to omit and it deadlocks you. If `draw_bitmap` fails, no completion callback will ever fire for that buffer, so its permit is never returned. Six failures and the pool is empty and the task blocks forever. Giving the permit back restores the invariant.

band_colour(band, frame)

Different colour per stripe, and it rotates each frame. This is not decoration — **it is the test instrument**. With six identical stripes, reusing a buffer mid-transfer produces identical output and the bug is invisible. With six different ones it is immediately obvious.

22.7 Phase 2 — build and run

```
idf.py build
idf.py -p COM6 flash monitor
```

KEY POINT

Expected

```
c I lab22: 2 buffers x 19200 B DMA-capable I lab22: 60 frames in 1512 ms -> 39.7 fps | fill
310 ms | done=360
```

Six clean horizontal colour bands, rotating each frame. `done` must be exactly 6 per frame — 360 after 60 frames. If it is not, a transfer was queued that never completed.

WARNING**Check your frame rate against the bus, not against this page**

[CEILING] This display cannot go faster than the SPI clock allows, and the arithmetic is worth doing before you believe any number:

```
c full frame = 240 x 240 x 2 bytes = 115,200 B = 921,600 bits at 40 MHz = 921,600 / 40,000,000
= 23.04 ms per frame CEILING = 1 / 0.02304 = 43.4 fps
```

That is a hard upper bound assuming the bus is busy 100% of the time. Real figures land below it, because six per-stripe window-set commands, CS transitions and driver overhead all cost time the arithmetic ignores.

An earlier edition of this text printed 57.5 fps here. That is 1.32× the ceiling and therefore impossible — the number was never measured, only made to look plausible. If your own log ever shows a rate above 43.4, do not celebrate: something is wrong with your timing, most likely a frame counter incrementing when no transfer occurred.

[MEASURE] Record your actual rate. Anything in the 30s is plausible at `NBUF = 2`.

22.8 Phase 3 — prove it

#	Do this	Expected
1	Run for 10 minutes.	No tearing, no stripe showing a neighbour's colour, <code>done</code> exactly 6× frames.
2	Reproduce the original bug. Set <code>NBUF</code> to 1 and delete the <code>xSemaphoreTake</code> line.	Visible corruption — stripes with wrong or blended colours — because the CPU overwrites the buffer mid-transfer.
3	Restore the take, keep <code>NBUF</code> at 1.	Correct again, but slower : no overlap is possible with one buffer.
4	Set <code>NBUF</code> back to 2.	Correct and faster. Record both frame rates.

KEY POINT**Case 2 is the debug-log entry**

Deliberately reintroducing a bug and watching it corrupt the display is far stronger evidence that you understand DMA buffer ownership than any amount of correct code. **"I proved the single-buffer version corrupts by giving each stripe a distinct colour"** is specific and verifiable.

22.9 Extensions, worked

NOTE**How numbers here are labelled**

[CEILING] a physical upper bound you can compute. **[DERIVED]** arithmetic, shown. **[MEASURE]** you must run it. Only **[MEASURE]** results are yours to claim.

22.9.1 — Measuring the overlap, properly

A faster wall-clock time alone does not prove overlap — it could be noise. Measure fill time and total time separately, which the listing already does via `t_fill`.

KEY POINT

Result and the reasoning

[MEASURE] Fill in this table from your own runs. No frame rates are printed here, because the previous edition printed invented ones and they were above the physical ceiling.

NBUF	Frame rate (yours)	Fill time / 60 frames (yours)	What it should show
1			fill and transfer are serial : total $\approx$ fill + transfer
2			fill is hidden inside transfer time

[DERIVED] What you are looking for is not simply "2 is faster." It is that **fill time stays roughly the same across both rows while the frame rate rises**. The CPU does identical work either way; what changes is whether that work happens *during* a transfer or *between* transfers.

If your fill time also changes materially, something else moved too and the comparison is not clean — check you changed only `NBUF`.

[CEILING] Neither row can exceed 43.4 fps. If `NBUF = 1` comes out close to `NBUF = 2`, your fill is much cheaper than your transfer, and the overlap has little left to hide. That is a legitimate result, not a failed experiment — say so rather than adjusting until it looks better.

For a definitive proof, toggle a GPIO high during the fill and watch it on the analyzer against the SPI clock. With `NBUF = 2` the fill pulse sits *inside* the clock burst; with 1 it sits between bursts. That capture is portfolio material.

22.9.2 — A full framebuffer in PSRAM

The alternative architecture: 112.5 KB is a serious chunk of 512 KB SRAM, but nothing against 8 MB of PSRAM.

```
static uint16_t *fb;
fb = heap_caps_malloc(H_RES * V_RES * sizeof(uint16_t),
                      MALLOC_CAP_SPIRAM | MALLOC_CAP_DMA);
configASSERT(fb);
// draw anywhere, compose freely, then send it all at once:
ESP_ERROR_CHECK(esp_lcd_panel_draw_bitmap(panel, 0, 0, H_RES, V_RES, fb));
```

KEY POINT**The trade, measured**

	Stripes (this lab)	Full FB in PSRAM
Internal SRAM used	37.5 KB	~0
Can read back what is on screen?	No	Yes
Draw at arbitrary coordinates?	awkward	trivially
Transfer speed	faster (internal RAM)	slower — PSRAM is reached over a serial bus

Real graphical firmware usually takes the full-framebuffer route because composing overlapping elements in stripes is painful. Knowing *why* both exist, and that the cost of PSRAM is bus bandwidth rather than capacity, is the point.

22.9.3 — If the colours are wrong

Symptom	Fix
Red shows as cyan, blue as yellow	<code>esp_lcd_panel_invert_color(panel, false)</code> — a genuine per-module difference
Red and blue swapped	<code>.rgb_ele_order = LCD_RGB_ELEMENT_ORDER_BGR</code>
Image shifted, coloured band at one edge	<code>esp_lcd_panel_set_gap(panel, x, y)</code> — many 240×240 modules are a 240×320 panel with part unused
Nothing at all, board otherwise fine	Backlight. Check BLK before anything else.

22.10 Docs and references

Topic	Where
LCD API and colour-transfer lifecycle	ESP-IDF v5.5 → Peripherals → LCD — states plainly that the caller owns the buffer until the done callback
ST7789 datasheet	Sitronix — the command set the panel driver encapsulates
DMA-capable allocation	ESP-IDF → System → Heap Memory Allocation, <code>MALLOC_CAP_DMA</code>

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

NOTE

Already fixed in the lab above — nothing to do

This section is the engineering record: what the earlier version of this lab got wrong, how it was found, and what replaced it. The lab as printed above is the corrected version. It is kept because the method used to catch the error is worth more than the error itself.

Lab 22 — the double buffering described was not implemented

WARNING

Confirmed architecture mismatch.

§22.3 and §22.5 say the CPU prepares the next stripe while DMA transmits the current one, and credit `.trans_queue_depth = 10` for the overlap. **The code allocates one buffer, fills it once before the loop, and queues six transfers from that same pointer.** Queue depth permits several transfers to be outstanding; it does not create application-level double buffering, and it does not give you a second buffer to fill.

Why it appears to work: every stripe contains identical data, so reusing the buffer while a transfer is in flight is harmless. Change one stripe's contents and the bug becomes visible immediately.

The API contract: ESP-IDF's LCD documentation states that colour transmission is queued and performed in the background, and that the *caller* owns the buffer's lifecycle — a buffer may be recycled only after `on_color_trans_done` fires for it.

Fix: Lab 22 implements genuine ping-pong: two DMA-capable buffers, an explicit FREE / FILLING / IN_FLIGHT state per buffer, and release driven by the transfer-done callback. It renders a different colour per stripe so that stale reuse is visible rather than hidden.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 22.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22 ←	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.